HACKATÓN EXPOASI 2026 · UNAL MANIZALES

SARD

Sistema Agéntico de Respuesta y Resiliencia ante Desastres

Documento de arquitectura del prototipo SARD: un sistema de coordinación de emergencias que opera sobre redes degradadas, agrupa reportes ciudadanos por lugar y tiempo, los prioriza por reglas verificables y deja la decisión final de despacho en manos de una persona.

ENTREGABLE

E3 — Arquitectura

Documento formal para el jurado

REPOSITORIO

Dfill-16/sard_hackaton

rama feat/capa-negocio · commit 0072a92

FECHA

10 de noviembre de 2026

PHP 8.5 · Laravel 13.40

00 Resumen ejecutivo

SARD coordina la respuesta a desastres cuando la infraestructura de comunicación habitual está caída. Recibe reportes ciudadanos en formato binario, los agrupa por proximidad espacial y temporal, asigna una prioridad calculada por reglas explícitas y presenta al operador una cola con la evidencia completa para que decida a qué unidad enviar. Si el nodo líder deja de responder, otro lo sustituye de forma automática sin duplicar ni perder la cadena de eventos.

El sistema se ejecuta como **cinco procesos del mismo código Laravel**, distinguidos únicamente por la variable de entorno `NODE_ROLE`. Cada proceso tiene su propia base SQLite, de modo que opera de forma independiente y necesita replicar por UUID. Los mensajes entre procesos atraviesan un *canal simulado* que pierde, retrasa y desordena mensajes de forma reproducible, para probar el sistema en condiciones hostiles.

COMPRESIÓN DEL REPORTE
78 B
78 bytes binarios frente a 185 bytes en JSON

DETECCIÓN DE CAÍDA
10 s
10 latidos perdidos de 1 s cada uno

PRUEBAS AUTOMÁTICAS
210
2316 aserciones, todas en verde

> **Cómo se verificó.** Cada dato del documento sale del código (con cita `archivo:línea`) o de una ejecución real con los cinco procesos levantados: reporte → ACK → triaje → decisión humana → alerta a campo → acuse, failover del líder con marca de tiempo e integridad de la cadena de hash. Las estimaciones y las propuestas futuras se rotulan como tales.

`Tabla 1` **Contenido requerido y dónde está**

REQUISITO DEL ENTREGABLE	SECCIÓN	FIGURAS
Diagrama de componentes	01	Fig. 1 y Fig. 2
Distribución de agentes	02	Fig. 3
Flujos de mensajes	03	Fig. 4
Almacenamiento	04	Fig. 5
Dependencias	05	Fig. 6

Alcance y límites del documento

Este documento describe **la arquitectura que existe en el código**. No cubre los entregables E5 (protocolo formal) ni E6 (informe de pruebas). El enlace por radiofrecuencia de la sección 11 es una propuesta posterior al prototipo y no forma parte de lo implementado.

01 Diagrama de componentes

El sistema gira en torno al **canal simulado**. Los mensajes de negocio (reporte, ACK, alerta y acuse) viajan siempre por él; el tráfico de control entre B y C (latidos, réplica y unión) y el anuncio de un nuevo líder son enlaces directos. Cada proceso guarda su cuaderno de eventos en su propia base SQLite.

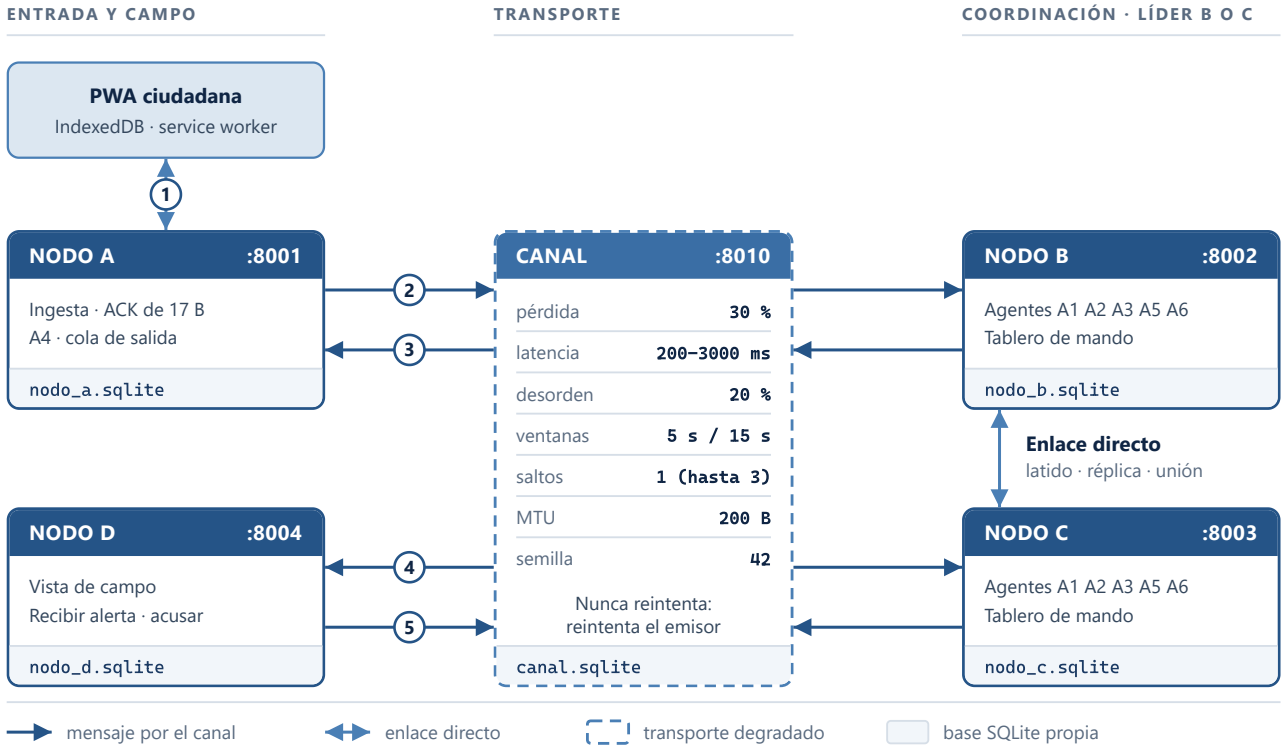


Fig. 1 — Componentes y flujos. Cinco procesos del mismo código, diferenciados por `NODE_ROLE`. El líder es B o C; el otro actúa como seguidor.

1. La PWA envía el reporte binario a A y recibe el ACK de 17 B.
2. A lo envía por el canal al líder.
3. El líder confirma con un ACK que vuelve a A por el canal.
4. Tras la decisión humana, el líder envía la alerta a D por el canal.
5. D devuelve el acuse de recibo al líder por el canal.

Fuente: `scripts/start.ps1:51–57,71–77` · `routes/api.php:23–82` · `routes/web.php:8–20`

01.1 Topología de replicación entre B y C

B y C ejecutan el mismo código y compiten por el papel de líder. El líder envía un **latido** directo al otro cada segundo; el seguidor que acumula diez segundos sin recibirlo sube la **época**, asume el mando y anuncia el cambio a A y D. El seguidor copia el cuaderno del líder por lotes. Cuando un nodo caído regresa, descubre la época mayor, queda como seguidor y une su cuaderno por UUID sin duplicar eventos.

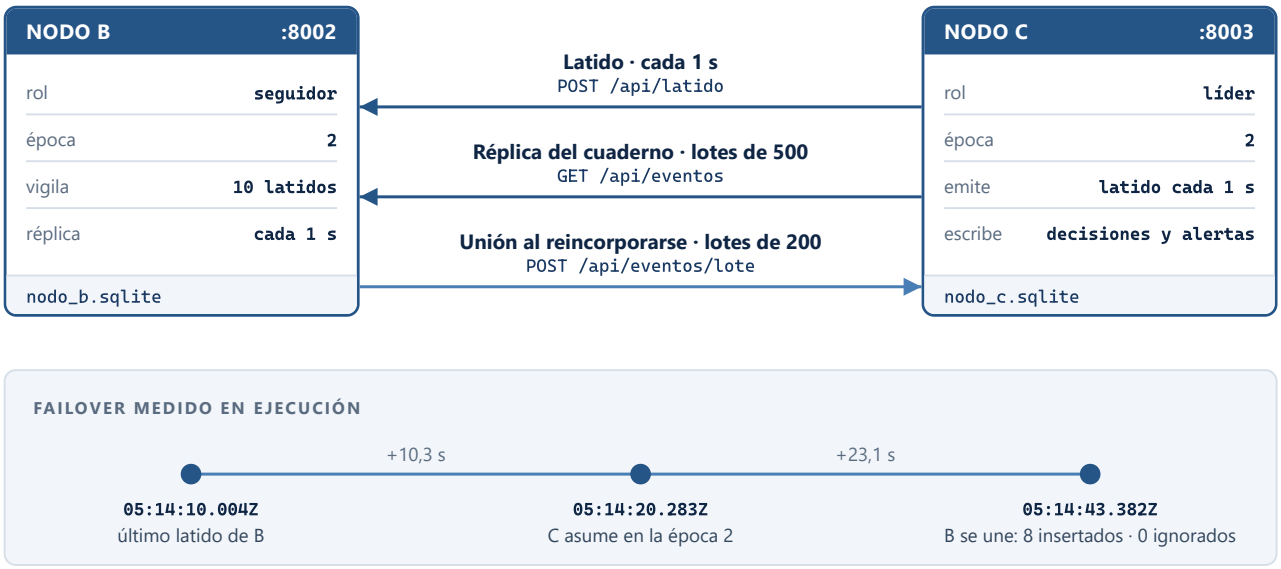


Fig. 2 — Replicación directa entre B y C, sin pasar por el canal. El latido lo emite solo el líder; el seguidor pide los eventos que le faltan y, al reincorporarse, un nodo envía al líder lo que solo él tiene. Un `409 epoca_vieja` indica que otro nodo ya asumió una época mayor.

`Fuente: app/Agents/A5/Resiliencia.php:15–23,86–112,212–253 · app/Node/B/ControlController.php:31–96 · config/sard.php:57–59`

02 Distribución de agentes

SARD tiene **seis agentes (A1 a A6)**. Son módulos del mismo código Laravel, no procesos aparte: cada proceso carga solo los que corresponden a su rol y los ejecuta con los trabajadores que lanza scripts/start.ps1. A1, A2, A3 y A6 corren únicamente en el nodo que es líder; A5 corre en B y C; A4, en el nodo A. D y el CANAL no ejecutan agentes.

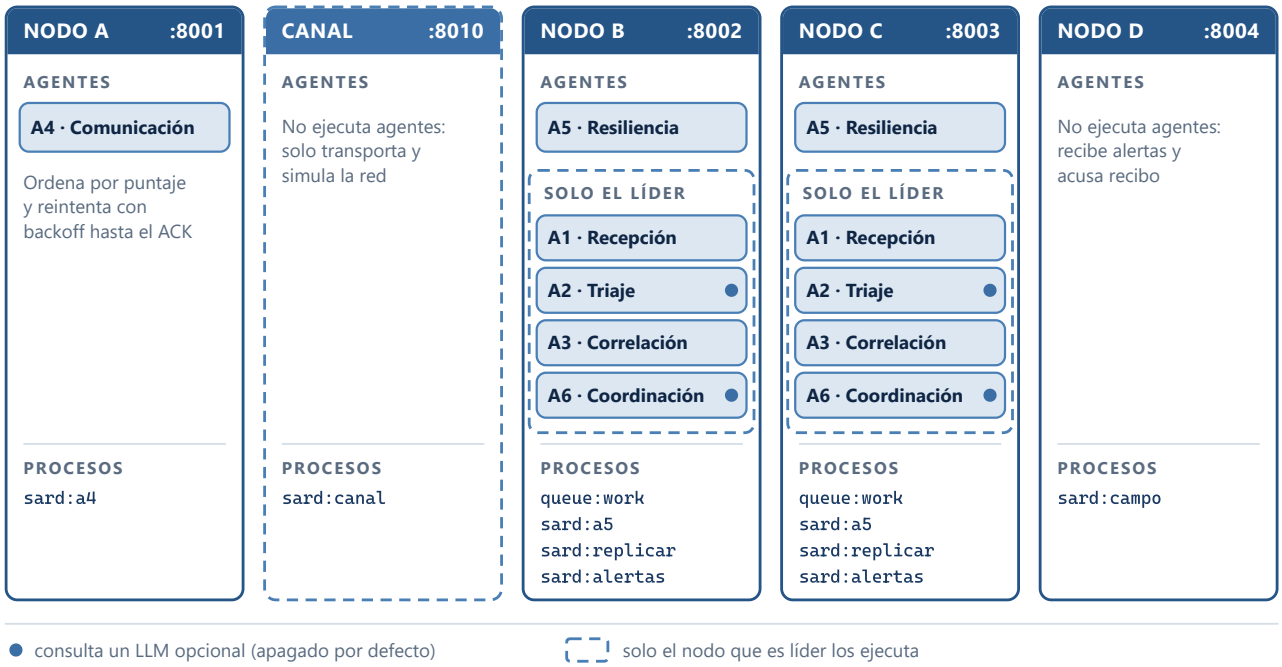


Fig. 3 — Agentes por proceso y trabajadores que los ejecutan. B y C cargan los mismos agentes; el seguidor no los ejecuta, solo replica el cuaderno y proyecta.

Fuente: scripts/start.ps1:44–57 · app/Agents/ · app/Jobs/ProcesarReporte.php:27–57

Tabla 2 **Responsabilidades de cada agente** app/Agents/

AGENTE	CORRE EN	QUÉ HACE	ENTRADA → SALIDA	LLM
A1 Recepción	líder	Normaliza el reporte decodificado sin inventar datos: sin GPS queda «sin ubicación», 0 personas es «desconocido», el texto dañado se conserva truncado	payload decodificado → `ReporteNormalizado`	no
A2 Triaje	líder	Asigna prioridad P1 a P4 con puntaje, señales e incertidumbre. Nada se descarta por prioridad	`ReporteNormalizado` → `TriajeAsignado`	opcional: solo puede subir la prioridad
A3 Correlación	líder	Agrupa reportes cercanos en espacio y tiempo (DBSCAN incremental: 300 m, 30 min, mismo tipo o bandera común) y declara incidente masivo desde 5 dispositivos distintos. Nunca fusiona ni borra originales	`TriajeAsignado` → `IncidenteAgrupado`	no
A4 Comunicación	nodo A	Decide qué reporte sale primero (mayor puntaje, menos bytes), envía lotes de 10 por el canal y reintenta con espera creciente hasta el ACK	`cola_salida` → mensajes al canal	no

AGENTE	CORRE EN	QUÉ HACE	ENTRADA → SALIDA	LLM
A5 **Resiliencia**	B y C	Emite y vigila latidos, asume el mando si el líder calla, avisa a A y D, reanuda lo pendiente y une cuadernos al regresar un nodo	latidos → `LiderAsumido`, `NodoReunido`	no
A6 **Coordinación**	líder	Propone una acción por incidente con plantillas y envía a D las alertas ya aprobadas, con reintento hasta el acuse. No despacha sin decisión humana	incidente → `RecomendacionPropuesta`; `DecisionHumana` → `AlertaCampoEnviada`	opcional: solo redacta la justificación

02.1 Orden de ejecución en el líder

A1 se ejecuta al recibir el reporte por el canal. Después, un trabajo de la cola de Laravel (`ProcesarReporte`) ejecuta A2, A3 y A6 en ese orden. El trabajo es idempotente: si el reporte ya tiene triaje o incidente, no repite esos eventos. Si el líder cae a mitad de camino, A5 en el nuevo líder reencola lo que quedó sin terminar.

> **Uso de modelos de lenguaje.** Solo A2 y A6 pueden consultar un LLM, y solo con `LLM_HABILITADO` (apagado por defecto). A2 únicamente puede subir una prioridad, nunca bajarla ni descartar; A6 únicamente redacta la justificación, sin cambiar la unidad ni la acción. Si el modelo falla, el sistema sigue por reglas.
>
> `Fuente: app/Agents/A2/ElevacionLlm.php:9–14 · app/Agents/A6/RedaccionLlm.php:7–11 · config/sard.php:62,72–81`

03 Flujos de mensajes

El recorrido completo de un reporte tiene nueve pasos, desde que el ciudadano lo envía hasta que el equipo de campo acusa recibo de la alerta. Las flechas oscuras viajan por el canal simulado y pueden perderse, retrasarse o desordenarse; las claras son HTTP directo.

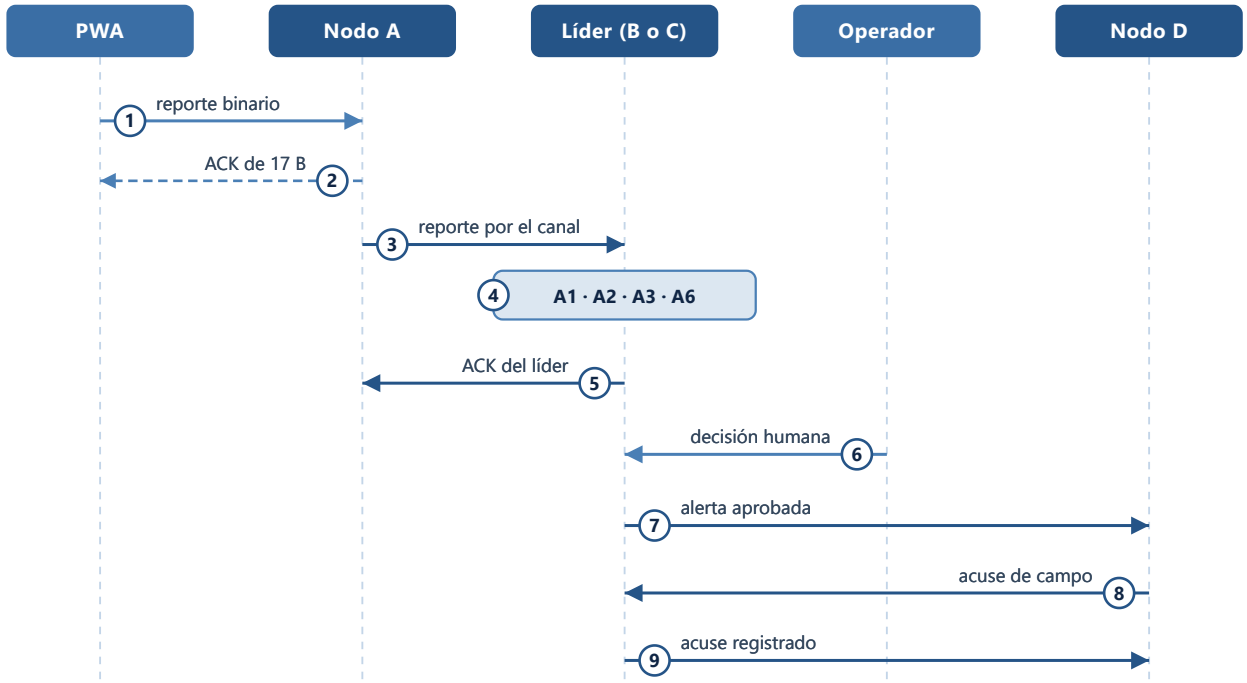


Fig. 4 — Flujo de extremo a extremo. El paso 4 agrupa lo que el líder hace por dentro con los agentes A1, A2, A3 y A6. Cada paso se detalla en la Tabla 3.

`Fuente: app/Node/A/ReportesController.php · app/Agents/A4/Comunicacion.php · app/Node/B/EntregaController.php · app/Agents/A6/EnvioAlertas.php`

Tabla 3 **Mensajes del flujo principal** routes/api.php:35–81

PASO	MENSAJE	ORIGEN → DESTINO	VÍA	EFECTO
1	`POST /api/reportes`	PWA → A	directo	A valida el CRC, guarda `ReporteRecibido` y encola el reporte en `cola_salida`
2	`ACK · 0x41 + UUID`	A → PWA	directo	Confirma la recepción en A; un reporte repetido recibe el mismo ACK
3	`POST /api/entrega/reporte`	A → líder	canal	A4 lo envía; el seguidor responde `409` `no_lider` y A reintenta
4	`eventos del cuaderno`	dentro del líder	local	`ReporteNormalizado`, `TriajeAsignado`, `IncidenteAgrupado` y `RecomendacionPropuesta`
5	`POST /api/acks`	líder → A	canal	A marca `confirmado_en` y deja de reintentar
6	`POST /api/decisiones`	operador → líder	directo	Registra `DecisionHumana`; si aprueba, nace `AlertaCampoEnviada`
7	`POST /api/entrega/alerta`	líder → D	canal	D la guarda en `alertas_campo`; el líder reintenta hasta el acuse
8	`POST /api/entrega/ack-campo`	D → líder	canal	El líder registra `AckCampo`; D reintenta hasta que se lo confirmen
9	`POST /api/entrega/ack-recibido`	líder → D	canal	D marca el acuse como confirmado (`confirmado_en`)

03.1 Mensajes fuera del flujo principal

- **Control entre B y C**, siempre directo: latido (`POST /api/latido`), réplica (`GET /api/eventos`) y unión (`POST /api/eventos/lote`), descritos en la sección 01.1.
- **Anuncio de nuevo líder** a A y D (`POST /api/lider`), directo, tras un failover.
- **Consulta del tablero** (`GET /api/tablero/estado`), que puede atender cualquier nodo B o C.

Todo mensaje que cruza el canal es idempotente: el UUID del reporte, de la alerta o del acuse permite repetirlo sin duplicar efectos. El comportamiento del canal se detalla en la sección 09.

04 Almacenamiento

Cada proceso guarda sus datos en **su propio archivo SQLite**; no hay servidor de base de datos. La fuente de verdad es el **cuaderno de eventos** (`eventos`): un registro de solo añadir con hash encadenado. Las demás tablas de negocio son *proyecciones* que el `Proyector` deriva de esos eventos, de modo que se pueden reconstruir en cualquier momento.

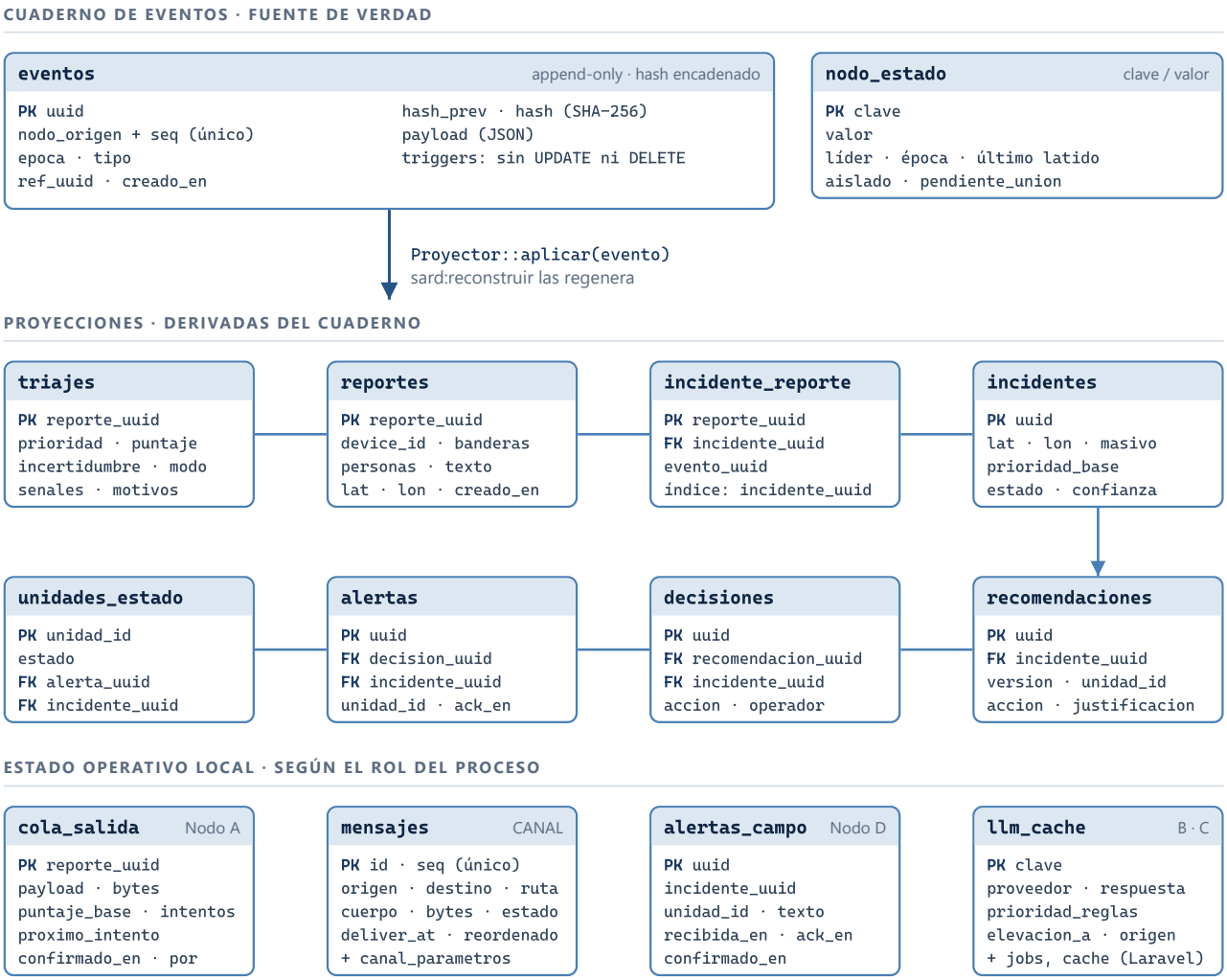


Fig. 5 — Esquema de almacenamiento. `PK` es clave primaria y `FK` es referencia por UUID a otra tabla (SQLite no declara claves foráneas; la relación la mantiene el Proyector). Los conectores unen las tablas relacionadas.

Fuente: `database/migrations/` · `app/Sard/Log/EventLog.php` · `app/Sard/Proyeccion/Proyector.php:20-39`

Tabla 4 **Base de datos de cada proceso** scripts/start.ps1:51-57,71-85

PROCESO	ARCHIVO	QUÉ GUARDA EN LA PRÁCTICA
Nodo A	`database/nodo_a.sqlite`	`eventos` (`ReporteRecibido`), `cola_salida` con el payload binario y el estado de cada reintento
CANAL	`database/canal.sqlite`	`mensajes` con su `deliver_at` y estado, y `canal_parametros` con los cambios en vivo
Nodo B	`database/nodo_b.sqlite`	Cuaderno completo (propio y replicado), todas las proyecciones, `llm_cache`
Nodo C	`database/nodo_c.sqlite`	Igual que B; es la réplica que asume el mando si B cae
Nodo D	`database/nodo_d.sqlite`	`alertas_campo` con el estado del acuse y sus reintentos

Todos los archivos comparten el mismo esquema, porque cada arranque ejecuta las migraciones, y cada rol usa solo su parte. También contienen `nodo_estado` (líder, época y último latido) y las tablas de Laravel para trabajos (`jobs`), caché y sesiones.

Tabla 5 **Eventos del cuaderno y su proyección** app/Sard/Proyeccion/Manejadores/

EVENTO	LO EMITE	TABLAS QUE ACTUALIZA
`ReporteRecibido`	nodo A	ninguna: queda en el cuaderno de A
`ReporteNormalizado`	A1	`reportes`
`TriajeAsignado`	A2	`triajes`
`PrioridadElevadaLLM`	A2 (opcional)	`triajes` , `incidente_reporte`
`IncidenteAgrupado`	A3	`incidentes` , `incidente_reporte`
`IncidenteMasivoDeclarado`	A3	`incidentes`
`RecomendacionPropuesta`	A6	`recomendaciones` , `incidentes`
`DecisionHumana`	tablero	`decisiones` , `incidentes`
`AlertaCampoEnviada`	A6	`alertas`
`AckCampo`	líder	`alertas`
`LiderAsumido · NodoReunido`	A5	ninguna: auditoría del failover

04.1 Garantías del almacenamiento

- **Solo añadir.** Dos triggers de SQLite abortan cualquier `UPDATE` o `DELETE` sobre `eventos` .
- **Cadena de hash.** Cada evento guarda `sha256(hash_prev|json canónico)` ; `php artisan sard:verificar-log` la recorre y detecta cualquier alteración.
- **Orden por origen.** La pareja `(nodo_origen, seq)` es única, por lo que cada nodo numera sus propios eventos sin coordinarse con otro.
- **Reconstrucción.** `php artisan sard:reconstruir` vuelve a derivar todas las proyecciones desde el cuaderno.
- **Réplica por UUID.** La unión inserta los eventos que faltan e ignora los que ya existen, sin duplicar.

05 Dependencias

El prototipo se ejecuta con PHP, Laravel y SQLite. No requiere Redis, un broker de mensajes, contenedores ni servicios en la nube. Lo único externo es la consulta opcional a un modelo de lenguaje, apagada por defecto.



Fig. 6 — Dependencias por capa. Las flechas indican quién llama a quién en tiempo de ejecución; la última capa solo interviene al construir y probar.

Fuente: `composer.json` · `package.json` · `config/sard.php:62,72–81`

Tabla 6 **Dependencias y versiones** `composer.json` · `package.json`

DEPENDENCIA	VERSIÓN	PARA QUÉ SE USA	NECESARIA
EJECUCIÓN DEL SERVIDOR			
`php`	`^8.3 (probado 8.5.12)`	Lenguaje de todos los nodos	sí
`laravel/framework`	`^13.17 (13.40.0)`	HTTP, rutas, colas, migraciones y cliente HTTP	sí
`SQLite · pdo_sqlite`	`incluido en PHP`	Base de datos de cada proceso, cola de trabajos, caché y sesiones	sí
`ext. sodium`	`incluida en PHP`	Firma Ed25519 de lotes entre B y C	solo con `SARD_FIRMA`
CLIENTE (NAVEGADOR)			
`alpinejs`	`^3.17.5`	Interactividad de PWA, tablero y campo	sí
`leaflet`	`^1.9.4`	Mapa del tablero	sí
`protomaps-leaflet`	`^5.1.0`	Dibuja los tiles vectoriales sobre Leaflet	sí

DEPENDENCIA	VERSIÓN	PARA QUÉ SE USA	NECESARIA
`pmtiles`	`^3.2.1`	Lee el archivo local public/tiles/manizales.pmtiles sin servidor de mapas	sí
SERVICIOS EXTERNOS			
`Gemini · OpenRouter · NVIDIA NIM`	`API compatible con OpenAI`	Cadena de tres proveedores para A2 y A6; las claves viven solo en `.env` y no se envía el `device_id`	no
CONSTRUCCIÓN Y PRUEBAS			
`vite · laravel-vite-plugin`	`^8.0 · ^3.1`	Compilan y versionan el JavaScript y el CSS en public/build	al construir
`tailwindcss · @tailwindcss/vite`	`^4.0.0`	Estilos de las pantallas	al construir
`phpunit/phpunit`	`^12.5.12`	210 pruebas con 2316 aserciones	pruebas
`laravel/pint · mockery · faker`	`^1.27 · ^1.6 · ^1.23`	Formato del código, dobles de prueba y datos sintéticos	desarrollo

La fuente tipográfica de las pantallas (Instrument Sans) se descarga al construir y se sirve desde public/build, por lo que el sistema funciona sin acceso a internet.

06 Especificaciones de componentes

06.1 Superficie REST

No existe un API Gateway independiente: la capa HTTP de Laravel cumple esa función. El acceso se resuelve por **rol de proceso** mediante middlewares, no por identidad de usuario. Las rutas se agrupan por nodo en routes/api.php.

`Tabla 7` **Endpoints REST por rol de nodo** `routes/api.php:23–82`

MÉTODO	RUTA	FUNCIÓN	ACCESO
TODOS LOS NODOS			
GET	`/api/salud`	Estado del nodo según su rol (A: cola pendiente; B y C: líder, época, último latido, seq_max)	sin middleware
POST	`/api/lider`	Un nuevo líder se anuncia ante A y D	rol A,D · fencing
NODO A · :8001			
POST	`/api/reportes`	Recibe el payload binario y responde el ACK de 17 B. `400` si es inválido, `413` si supera 176 B	solo A · fencing
POST	`/api/acks`	El líder confirma, por el canal, que recibió el reporte (idempotente)	solo A · fencing
GET	`/api/metricas`	Reportes recibidos y ahorro de bytes frente a JSON	solo A · fencing
NODO D · :8004			
GET	`/api/campo/alertas`	Alertas recibidas por el equipo de campo	rol D · fencing
POST	`/api/campo/ack`	El equipo de campo acusa recibo de una alerta	rol D · fencing
POST	`/api/entrega/alerta`	Alerta entregada por el canal	rol D · fencing
POST	`/api/entrega/ack-recibido`	El líder confirma, por el canal, que registró el acuse	rol D · fencing
CANAL · :8010			

MÉTODO	RUTA	FUNCIÓN	ACCESO
POST	/api/canal/enviar	Encola un mensaje en base64 y responde 202 con id y estado. 413 si supera la MTU de 200 B	rol CANAL
GET	/api/canal/estadisticas	Recibidos, perdidos, entregados, reordenados, fallidos y pérdida por salto	rol CANAL
POST	/api/canal/config	Cambia pérdida, ventanas y saltos en vivo. 403 sin SARD_DEMO	rol CANAL
NODOS B Y C · :8002 / :8003			
POST	/api/entrega/reporte	El canal entrega un reporte de A. 409 no_lider si el nodo es seguidor	rol B,C · fencing
POST	/api/entrega/ack-campo	Acuse de campo recibido en el líder; el seguidor responde 409	rol B,C · fencing
GET	/api/tablero/estado	Cola priorizada, contadores, recursos y brechas	rol B,C · fencing
GET	/api/incidentes/{uuid}	Detalle del incidente con toda su evidencia	rol B,C · fencing
POST	/api/decisiones	Aprobar o descartar una recomendación. 409 no_lider en el seguidor	rol B,C · fencing
POST	/api/latido	Latido del líder. 409 si la época es vieja	rol B,C · fencing
GET	/api/lider	Quién lidera y en qué época	rol B,C · fencing
GET	/api/eventos	Eventos ordenados por (nodo, seq) desde un punto dado; base de la réplica	rol B,C · fencing
POST	/api/eventos/lote	Unión de cuadernos por UUID. 401 si la firma del lote no cuadra	rol B,C · fencing
POST	/api/reportes/faltantes	Cuáles de los UUID enviados (máximo 500) no están en este nodo	rol B,C · fencing
POST	/api/demo/aislar	Aísla o reactiva el nodo para demostrar el failover. Requiere SARD_DEMO	rol B,C · fencing

Restricción de rol. SoloNodoA y SoloRol responden 404 (no 403) cuando el rol no corresponde, de modo que las rutas de un rol no existen para los demás. El fencing responde 409 epoca_vieja si la cabecera X-Sard-Epoca trae una época menor que la local.

Fuente: app/Http/Middleware/SoloNodoA.php:17-23 · SoloRol.php:17-23 · FencingMiddleware.php:18-29

06.2 Superficie web

Tabla 8 **Pantallas por nodo** routes/web.php:16-20

RUTA	NODO	PUERTO	ACCESO	USUARIO Y PROPÓSITO
/pwa	A	8001	solo A	Ciudadano: formulario mínimo y cola offline en el teléfono
/tablero	B o C	8002 / 8003	rol B,C	Coordinador: cola priorizada, mapa, evidencia y decisión
/campo	D	8004	rol D	Equipo de campo: lista de alertas y acuse de recibo

07 Payload binario v1

El reporte ciudadano viaja como un paquete binario de **36 + n bytes** (n es el largo del texto, de 0 a 140). Se arma en el navegador con JavaScript y se lee en el servidor con PHP; ambos codecs producen y aceptan exactamente los mismos bytes. Todos los enteros son *big-endian*.

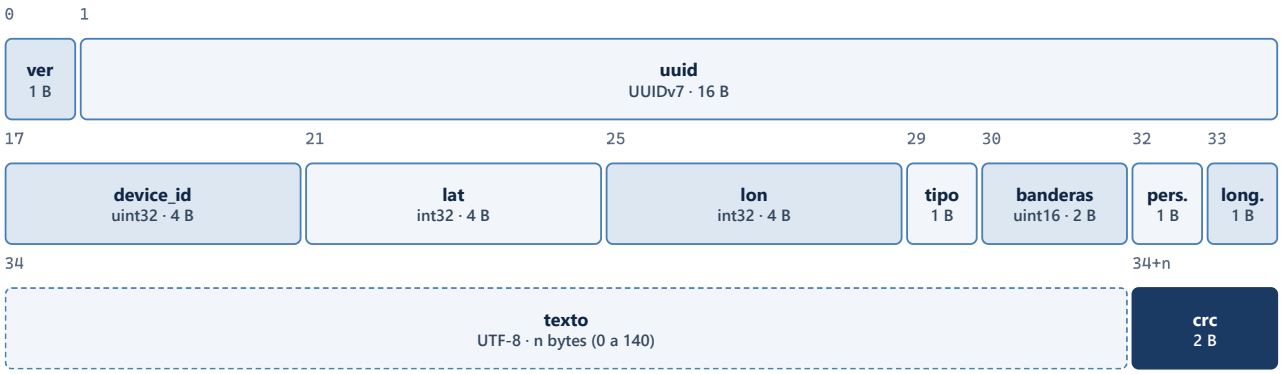


Fig. 7 — Mapa de bytes del payload. Cada columna es un byte y los números sobre los campos son su offset. `ver`, `pers.` y `long.` abrevian versión, personas y largo_texto.

Fuente: app/Protocol/PayloadReporte.php:7–29 · resources/js/protocol/payload.js:71–160

Tabla 9 **Campos del payload v1** PayloadReporte.php:7–29

OFFSET	BYTES	CAMPO	TIPO	REGLA
0	1	versión	uint8	Siempre 1; otro valor se rechaza
1	16	uuid	UUIDv7	Canónico 8-4-4-4-12, versión 7 y variante RFC 4122
17	4	device_id	uint32	Anónimo, aleatorio por instalación
21	4	lat	int32	Grados × 10^7; `0x7FFFFFFF` significa sin ubicación
25	4	lon	int32	Igual que `lat`
29	1	tipo	uint8	0 a 7
30	2	banderas	uint16	Bits del catálogo de banderas
32	1	personas	uint8	0 a 255; 0 significa desconocido
33	1	largo_texto	uint8	0 a 140, medido en **bytes** UTF-8
34	n	texto	UTF-8	Como máximo 140 bytes; debe ser UTF-8 válido
34+n	2	crc	uint16	CRC-16/CCITT-FALSE sobre los bytes 0 a 33+n

MÍNIMO
36 B
Reporte sin texto (`LARGO_MINIMO`)

MÁXIMO
176 B
36 + 140 bytes de texto; el nodo A rechaza más con 413

ACK
17 B
`0x41` seguido de los 16 bytes del UUID

07.1 Decisiones de diseño

- **Enteros en lugar de decimales.** La latitud y la longitud se guardan como grados × 10^7 en un int32: −75,5002 se almacena como −755 002 000. Cuatro bytes alcanzan para ±180°.
- **Un centinela para «sin ubicación».** Se usa `0x7FFFFFFF`, el mayor int32. No sirve 0 (es un punto real en el mar) ni −1 (es una longitud válida).
- **El texto se mide en bytes.** «vía bloqueada por derrumbe» tiene 26 caracteres y 28 bytes. Contar caracteres dejaría pasar textos con acentos que desbordan el paquete.

Tabla 10 **Tamaño frente a JSON** — medido en ejecución

FORMATO	REPORTE DE PRUEBA	1616 REPORTES
Binario v1	78 B	96 058 B
JSON equivalente	185 B	299 304 B
Diferencia	2,4 veces	3,1 veces

07.2 Dos lenguajes, un solo formato

Tres fixtures congelados en tests/fixtures/ guardan el mismo reporte codificado por PHP y por JavaScript en hexadecimal. Las pruebas de cada lenguaje decodifican el fixture del otro y nunca lo regeneran: si un codec cambiara un solo byte, la prueba fallaría en lugar de adaptarse.

08 CRC-16 y verificación

Los dos últimos bytes del payload son un CRC-16 que detecta **corrupción accidental** durante el transporte. El nodo receptor lo recalcula sobre los bytes 0 a 33+n y descarta el paquete si no coincide.

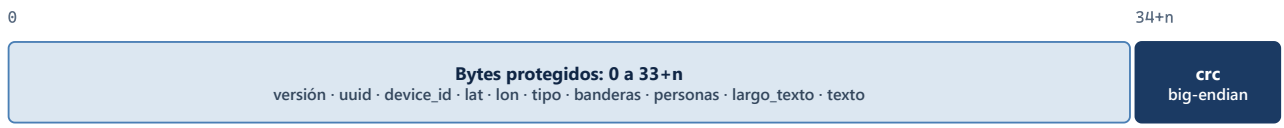


Fig. 8 — Cobertura del CRC. Se calcula sobre todo el paquete excepto sus propios dos bytes.
Fuente: app/Protocol/PayloadReporte.php:20–26,102–124,266–289

Tabla 11 **Parámetros del CRC** — PayloadReporte.php:45–47,266–289

PARÁMETRO	VALOR	NOTA
Variante	CRC-16/CCITT-FALSE	Estándar de uso común
Polinomio	0x1021	
Valor inicial	0xFFFF	
Reflexión de entrada y salida	no	
XOR final	0x0000	
Vector de prueba	"123456789" → 0x29B1	Oráculo independiente en tests/Unit/PayloadReporteTest.php:25

08.1 Orden de validación al decodificar

Tabla 12 **Comprobaciones de** `PayloadReporte::decode` `PayloadReporte.php:102–160`

PASO	COMPROBACIÓN
1	Largo mínimo de 36 bytes
2	Versión igual a 1
3	CRC recalculado igual al recibido
4	`largo_texto` menor o igual a 140
5	Largo total igual a 36 + `largo_texto`
6	`tipo` entre 0 y 7
7	UUID de versión 7 y variante RFC 4122
8	Texto en UTF-8 válido

Cualquier comprobación fallida lanza `PayloadInvalidoException`: el nodo responde 400, registra el motivo y no emite ACK.

Tabla 13 **Respuestas de** `POST /api/reportes` `ReportesController.php:34–73`

CÓDIGO	CAUSA	EFECTO
`200`	Reporte válido y nuevo	Se guarda en `eventos` y `cola_salida` en una transacción; responde el ACK de 17 B
`200`	UUID ya recibido	Devuelve el mismo ACK sin insertar de nuevo (idempotente)
`400`	Payload inválido (Tabla 12)	`payload_invalido`; se registra el motivo y no se emite ACK
`413`	Más de 176 bytes	`payload_demasiado_grande`; se comprueba antes de decodificar

> **Alcance del CRC.** Detecta un bit cambiado por ruido, pero no una manipulación deliberada: quien altere el contenido puede recalcular el CRC. La integridad frente a un adversario la dan la cadena de hash del cuaderno y la firma opcional entre B y C (sección 10).

09 Canal de mensajes simulado

El proceso CANAL representa una red hostil: pierde, retrasa y desordena mensajes entre A, B, C y D. La aleatoriedad es **reproducible**: la misma semilla produce la misma secuencia de pérdidas, de modo que una falla observada puede repetirse. El canal nunca reintenta; reintenta quien envía, y por eso el ACK tiene significado.

Tabla 14 **Parámetros del canal** config/sard.php:30–40

VARIABLE	VALOR	EFECTO
CANAL_PERDIDA	0,30	Probabilidad de perder un mensaje en cada tramo
CANAL_LAT_MIN_MS	200 ms	Latencia mínima de un tramo
CANAL_LAT_MAX_MS	3000 ms	Latencia máxima de un tramo
CANAL_DESORDEN	0,20	Probabilidad de que un mensaje llegue tarde y fuera de orden
CANAL_VENTANA_ABIERTA_S	5 s	Tiempo con el canal abierto en cada ciclo
CANAL_VENTANA_CERRADA_S	15 s	Tiempo con el canal cerrado; la entrega se pospone a la próxima apertura
CANAL_MTU_B	200 B	Tamaño máximo de un mensaje; más grande se rechaza con 413
CANAL_SEMILLA	42	Semilla del generador reproducible
CANAL_SALTOS	1	Tramos que atraviesa un mensaje; de 1 a 3

Pérdida, ventanas y saltos se pueden cambiar en vivo con `POST /api/canal/config` cuando `SARD_DEMO` está activo.

09.1 Cómo se decide cada fallo

Tabla 15 **Modelo de fallos** Canal.php:88–110,184–205 · Aleatorio.php:21–24

EFECTO	REGLA
Pérdida	El mensaje se pierde si `u(1)` es menor que la probabilidad de pérdida
Latencia	Retardo = mínimo + `u(2)` × (máximo − mínimo)
Desorden	Si `u(3)` es menor que la probabilidad de desorden, se suma un retardo de entre una y dos veces la latencia máxima (3 a 6 s)
Ventanas	El canal está cerrado cuando (ahora − arranque) mod (abierta + cerrada) es mayor o igual que la ventana abierta
Saltos	Cada tramo repite el sorteo con `u(k + 4·(s − 1))`; un mensaje puede perderse en cualquier tramo. Con un salto la secuencia es idéntica a la del canal sin multisalto

La función `u(k)` toma los primeros 4 bytes de `sha1(semilla|seq|k)` y los normaliza a [0, 1]. Depende solo de la semilla, del número de secuencia del mensaje y de k.

09.2 Protocolo de envío y confirmación

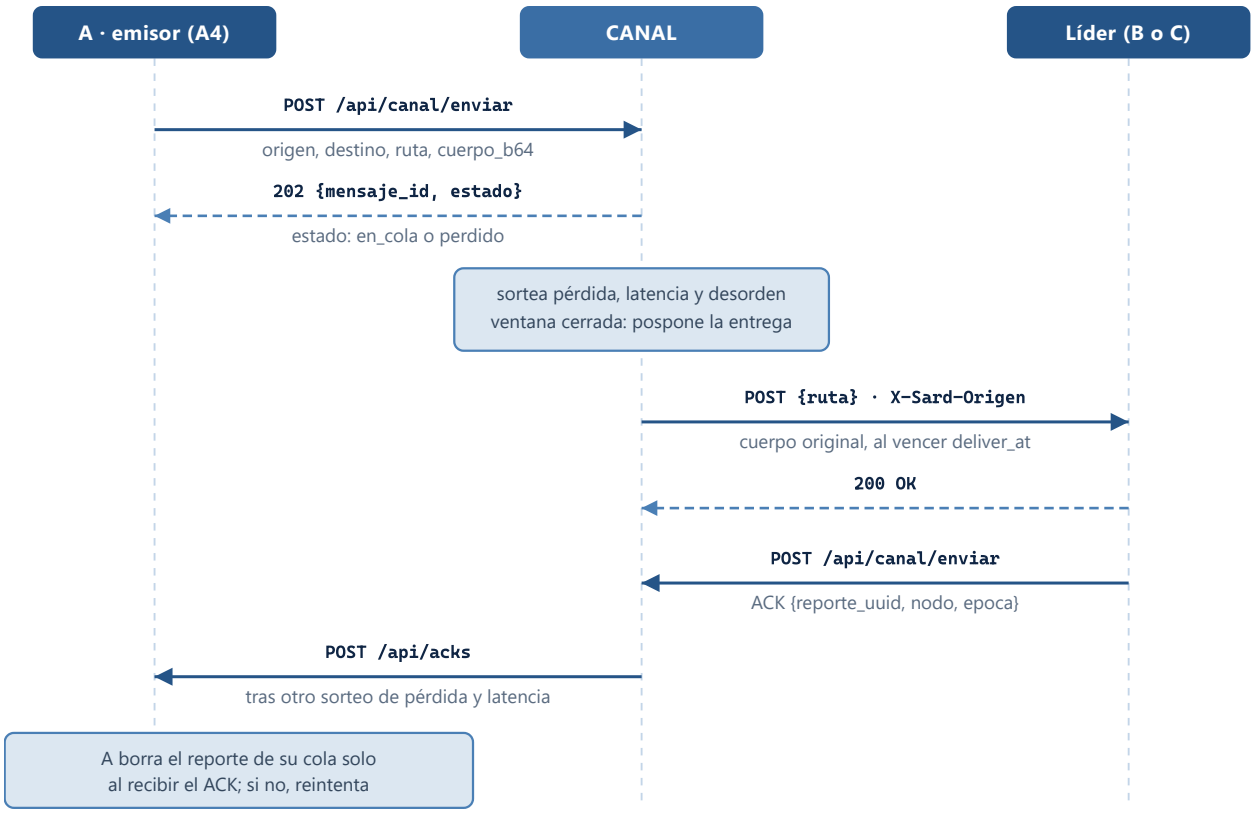


Fig. 9 — Envío de un reporte de A al líder y confirmación de vuelta. Cada trayecto por el canal puede perder, retrasar o desordenar el mensaje.
`Fuente: app/Channel/CanalController.php:17-43 · Canal.php:43-81,115-160 · app/Node/B/EntregaController.php:38-72`

09.3 Reintentos del emisor

A4 reintenta cada reporte hasta recibir el ACK, sin límite de intentos. La espera crece según una serie fija y se le aplica una variación aleatoria de ±20 % para que dos reportes que fallaron a la vez no reintenten en el mismo instante.

`Tabla 16` **Espera entre intentos** `Comunicacion.php:247-256 · config/sard.php:42-43`

INTENTO	1	2	3	4	5	6 O MÁS
Espera base	1 s	2 s	4 s	8 s	16 s	30 s

Como el UUID identifica cada reporte, un reintento que llega después de un ACK perdido no crea un duplicado: el nodo responde el mismo ACK (Tabla 13).

10 Seguridad y consistencia

Las garantías del sistema son de **consistencia e integridad**: que no haya dos líderes escribiendo, que el cuaderno no se altere y que ningún despacho ocurra sin una persona. No hay autenticación de usuarios, y el documento lo declara en la sección 10.3.

Tabla 17 **Mecanismos de consistencia e integridad** archivo:línea

MECANISMO	QUÉ GARANTIZA Y CÓMO	CÓDIGO
Fencing por época	Una escritura con `X-Sard-Epoca` menor que la local se rechaza con `409` `epoca_vieja`. Un líder viejo no puede escribir tras un failover	`FencingMiddleware.php:18-29`
Rutas por rol	Las rutas de un rol responden 404 en los demás nodos; un nodo solo expone lo que le corresponde	`SoloRol.php:17-23`
Latido y failover	El líder late cada 1 s. El seguidor asume tras 10 latidos perdidos (10 s), sube la época, anuncia el cambio y reencola lo que el líder anterior no terminó	`Resiliencia.php:86-165`
Cuaderno append-only	Dos triggers de SQLite abortan cualquier `UPDATE` o `DELETE` sobre `eventos`	`create_eventos_table.php:33-47`
Cadena de hash	Cada evento guarda `sha256(hash_prev|json canónico)`; el primero parte de 64 ceros. El hash de un evento remoto se verifica antes de insertarlo	`EventLog.php:113-143,260-273`
Idempotencia por UUID	Un reporte repetido recibe el mismo ACK sin insertarse; la unión de cuadernos ignora los UUID que ya existen	`ReportesController.php:63-67`
Firma Ed25519	El lote que viaja entre B y C va firmado y se verifica antes de insertar; si un byte no cuadra se rechaza el lote completo con `401`. **Apagada por defecto** (`SARD_FIRMA=false`)	`Firmador.php:35-60` · `ControlController.php:98-117`
Decisión humana	Ninguna alerta sale sin una `DecisionHumana`. Solo el líder acepta decisiones; el seguidor responde `409` `no_lider`	`Coordinacion.php:17-18` · `DecisionController.php:25-30`

10.1 Respuesta ante fallas

`Tabla 18` **Qué ocurre ante cada falla**

FALLA	RESPUESTA DEL SISTEMA
Mensaje perdido en el canal	A reintenta con espera creciente hasta recibir el ACK
ACK perdido	El reintento de A recibe el mismo ACK; no se duplica el reporte
Payload corrupto	El CRC no cuadra, el nodo responde 400 y descarta el paquete
Cae el líder	A los 10 s el seguidor asume con una época mayor y retoma el procesamiento pendiente
Un líder viejo sigue escribiendo	Recibe 409 `epoca_vieja` y redescubre al líder vigente
Regresa el nodo caído	Descubre la época mayor, queda como seguidor y une su cuaderno por UUID
Reporte o decisión en un seguidor	Responde 409 `no_lider`; el emisor reintenta

10.2 Verificación del failover

En una ejecución con los cinco procesos, el último latido de B llegó a las 05:14:10.004Z. C asumió como líder a las 05:14:20.283Z, es decir, 10,3 s después. B se reincorporó a las 05:14:43.382Z con 8 eventos insertados y 0 ignorados (Fig. 2).

10.3 Límites de seguridad

> **Sin autenticación de usuarios.** El sistema no tiene usuarios, contraseñas ni tokens. Los nodos se identifican por las cabeceras `X-Sard-Nodo` y `X-Sard-Epoca` y por el rol configurado en el entorno. Cualquier persona con acceso a la red local puede aprobar alertas o hacerse pasar por un nodo, y el fencing no detiene a un impostor que declare una época alta. Es coherente con el prototipo, que opera con datos sintéticos en una red controlada, pero un despliegue real exigiría autenticación entre nodos y por operador.

- El CRC no es una medida de seguridad (sección 08).
- La firma Ed25519 cubre solo el tráfico entre B y C, y requiere activar `SARD_FIRMA` y generar las llaves con `php artisan sard:llaves`. Las llaves privadas viven en **storage/keys/**, fuera de git.
- El modelo `User` de Laravel existe en el repositorio, pero ninguna ruta lo usa.

11 Escalabilidad a radiofrecuencia

> **Propuesta posterior al prototipo.** Ninguna parte del código actual implementa un enlace de radio. Esta sección describe cómo se podría extender SARD sin cambiar la lógica del backend. Los alcances son rangos generales de tecnologías de radio existentes, no mediciones del proyecto.

11.1 El problema

En un apagón real no hay internet de operadora ni siempre electricidad, y el celular no puede llegar al sistema. El prototipo asume una red IP local (Wi-Fi o cable) entre el teléfono y los nodos. La propuesta añade un enlace de radio de largo alcance, por ejemplo LoRa, con estaciones intermedias que no dependen de torres celulares ni de cableado.

11.2 Arquitectura propuesta



Fig. 10 — Cadena propuesta de comunicación. La estación y el gateway son la única capa nueva; a partir del nodo A el flujo es el del prototipo.
`Fuente: nodo A en routes/api.php:35–39`

1. El ciudadano reporta desde la PWA y el celular envía el paquete por Wi-Fi local a la estación más cercana.
2. La estación lo reenvía por radio hacia el gateway, directamente o a través de otras estaciones.
3. El gateway extrae el payload v1 y lo envía como cuerpo binario a `POST /api/reportes` del nodo A. Devuelve el ACK de 17 B por la misma vía.
4. Desde el nodo A el flujo es el descrito en las secciones 01 y 03: canal, líder, triaje y tablero.

`Tabla 19` **Qué cambia y qué se conserva**

SE CONSERVA (LÓGICA DE LARAVEL)	CAMBIA (CAPA DE TRANSPORTE)
Guardado de reportes con UUID e idempotencia	De HTTP directo celular → nodo a celular → estación → radio → gateway → HTTP → nodo
Triaje y priorización P1 a P4	Aparece el gateway como traductor entre radio y HTTP
Cambio de líder por época	Hay que definir una trama de radio con origen, destino, tipo de mensaje y checksum
Tablero y decisión humana	Cada estación necesita energía independiente (batería y panel solar)
Cuaderno append-only con cadena de hash	El gateway reenvía el payload sin convertirlo a JSON y devuelve el ACK

11.3 Lo que el prototipo ya anticipa

El canal simulado fue diseñado con las limitaciones de un enlace de radio. La Tabla 20 relaciona cada parámetro del prototipo con lo que implicaría en el enlace real.

Tabla 20 **Parámetros del prototipo frente al enlace de radio**

PARÁMETRO	PROTOTIPO	IMPLICACIÓN PARA EL ENLACE DE RADIO
Tamaño del payload	36 a 176 B	La trama de radio debe transportarlo entero. Con la MTU de 200 B como referencia, quedan 24 B para direcciones, tipo y checksum. Hay que confirmarlo con el hardware elegido
Pérdida	30 %	El ACK y el reintento con espera creciente ya cubren la pérdida; el UUID evita duplicados al retransmitir
Latencia y desorden	0,2 a 3 s	Las operaciones son idempotentes y no dependen del orden de llegada
Ventanas	5 s / 15 s	Representan un enlace intermitente; A4 conserva el reporte hasta confirmar
Saltos	1 a 3	Equivalen a estaciones repetidoras; cada tramo suma su propia pérdida
Integridad	CRC-16	El payload mantiene su propio CRC sobre cualquier control de errores de la capa de radio

11.4 Alcances estimados

Tabla 21 **Alcances aproximados** estimados, no del prototipo

CONEXIÓN	ALCANCE	COMENTARIO
Celular → estación (Wi-Fi local)	100 a 300 m	Depende de obstáculos y potencia del Wi-Fi
Estación → estación (radio)	2 a 20 km	Depende de frecuencia, antena y terreno
Estación en punto alto → ciudad	Amplia	Útil en ciudades montañosas como Manizales

11.5 Trabajo pendiente

1. Elegir el hardware de radio: módulos, frecuencia y potencia.
2. Definir la trama que viaja por el aire: dirección de origen y destino, tipo de mensaje y checksum.
3. Construir el gateway en cualquier lenguaje con puerto serie y HTTP. Debe reenviar el payload binario v1 sin convertirlo a JSON y devolver el ACK de 17 B.
4. Probar en campo alcances, consumo de batería y comportamiento sin electricidad.
5. Documentar el despliegue: ubicación de estaciones, permisos en postes y mantenimiento.

Con puntos estratégicos como una plaza, una escuela o un puesto de salud se cubre buena parte de un pueblo, sin una estación por vivienda.